

Who Can Turn Off Your Ethereum App?

A Full-Stack Walk-Away Test

James Carnley

Ethereum File System



Building Cypherpunk Apps

Why make an app you can walk away from?

What happens when an app disappears?

- Users lose their work and trust
- Other developers can’t rely on that functionality
- Communities splinter

Absence and Hostile Control

The team disappears

Can users continue without its services or help?

Can multiple operators run critical features?

The keys change hands

Can users refuse a harmful change?

Can the community choose the next version?

Benefits of walking away

Users can truly rely on your app

Run their lives

Keep their critical data in your app

Other developers can build on your app

We can build hyperstructures

You have a larger impact

When you can walk away, everyone benefits

The application stack

Interface

- DNS
- Web Hosting
- App Stores
- Accounts

Data

- Storage
- State
- Computation

Rules

- Governance
- Network and settlement
- Community



Interface

Finding and using the app

Find the accepted release

Run it without your servers

Recover access without your help

Can people find and use your app?

Names and entry points

- ENS names an app release on IPFS
- A gateway is one route to that release
- <EFS.eth points to IPFS web bundle><.limo or .link gateways work>

<diagram of DNS + HTTP vs ENS + IPFS>

The code that actually runs

- Ledger Connect Kit exposed a runtime dependency.
- A retained release includes what it loads later.
- Rebuilding also needs source and build inputs.

Accounts and Recovery

Safe: two of three independent signers

Two remaining signers can replace a lost one.

File decryption needs separate recovery.



Data Portability

Continuing the work

Is the data portable?

Keep usable files

Check the records

Replace the services doing the work

Data that remains available

- Data should be content addressed
- Multiple operators store data
- Standard formats and encryption keep those bytes useful
- <can we make a diagram or have an image for this?>

File verification with EFS

- An accepted file reference supplies a digest.
- IPFS and Arweave can carry the same file.
- A mismatch differs from a failed retrieval.

State you can rely on

- Use multiple RPC services
- Verify data via light clients like Helios
- Fresh: the state is recent enough
- Complete: the task has all required records

The work outside the contracts

PoolTogether V5 rewards permissionless claimers.

Replacement needs inputs, gas and an incentive.

Unclaimed prizes can expire.



Upgrades and Community

Coming to consensus on the rules

Who can change shared rules?

What can users refuse?

Can they keep using the rules they chose?

Who upgrades?

Rules and change authority

User-chosen software

Can users keep a release or switch clients?

Shared state may still change.

Shared application rules

Who can upgrade, pause or redirect them?

What can a user refuse or leave?

The network beneath the app

Connectivity and L1

Peers and node access

Inclusion and finality

Required chain history

L2 and settlement

Sequencer and proving

Available chain data

A tested recovery route

Operators and continuity

Hic et Nunc’s website closed. Community mirrors followed.

Usable code and retained artwork made continuation possible.

Independent operators still need resources.

Continuing somewhere else

Safe CLI offers a route without the hosted API.

The same account still needs signers and gas.

A usable alternative preserves the user’s task.



Putting this into practice

A walk away test for your app

A walk away test for your app

Choose one task whose loss would hurt.

Try it without your team's services or help.

Test what hostile authority could change.

Fix the dependency that causes the most harm.

Our work can outlive us


Thank you!
Let’s build reliable and long-lasting apps together
James Carnley
Ethereum File System
efs.eth.limo
JamesCarnley@gmail.com
@JamesCarnley